



ESCUELA SUPERIOR POLITÉCNICA DEL LITORAL

Facultad de Ingeniería en Electricidad y Computación

Sistemas de Bases de Datos Fase Final

PEGASO

AIRLINES

Sistema Integral de Gestión Aeronáutica

Velocidad y Confianza en la Aviación

Grupo: No. 6

Integrantes: Santiago Rubén Gómez Medina
Fanny Marcela López Ramos
José Luis Paladines Sánchez
Bruno Alejandro Román Torres
Matías Ariel Sánchez Baldeón

Profesora: Ph.D. Patricia Leonor Suárez Riofrío

Paralelo: 4

Término: I PAO 2026

Guayaquil, Ecuador 15 de Agosto de 2026

Índice

1. Nombre del Proyecto	2
2. Nombre de la Aplicación	2
3. Objetivo General	2
4. Funcionalidades del Sistema	3
4.1. Módulo de Reservasiones (7 funcionalidades)	3
4.2. Módulo de Check-in y Equipaje (5 funcionalidades)	3
4.3. Módulo de Vuelos y Tripulación (5 funcionalidades)	4
4.4. Módulo de Administración (5 funcionalidades)	5
5. Modelo Físico de Base de Datos	5
5.1. Diagrama Entidad-Relación	5
5.2. Organización de las 22 Tablas	7
5.3. Restricciones y Constraints Destacados	7
6. Implementación Funcional (Frontend)	9
6.1. CRUD 1: Gestión de Reservasiones	10
6.1.1. Cálculo Dinámico de Precios	10
6.2. CRUD 2: Check-in y Equipaje	11
6.3. CRUD 3: Gestión de Vuelos y Tripulación	13
6.4. Módulo de Administración	14
7. Video de Demostración Funcional	15
8. Procedimientos Almacenados	15
8.0.1. Ejemplo: sp_emitir_boleto (Control de Sobreventa)	16
8.0.2. Ejemplo: sp_checkin_pasajero (Validación de Clase)	17
9. Disparadores (Triggers)	17
9.0.1. Ejemplo: Protección contra eliminación física	18
10.Consultas SQL y Vistas	18
10.1. Vistas Implementadas (6)	18
10.1.1. Ejemplo: Vista de Ocupación de Vuelos	18
10.2. Índices Optimizados (10) y Consultas de Aplicación	19
11.Organización y Documentación del Proyecto	19
11.1. Estructura de Carpetas	20
11.2. Convenciones de Nomenclatura	20
11.3. Data Semilla	20
12.Conclusiones	21

1. Nombre del Proyecto

Identificación del Proyecto

Nombre del Proyecto: Proyecto Pegaso: velocidad y confianza en la aviación

El nombre **Pegaso** evoca al caballo alado de la mitología griega, símbolo de velocidad y elevación, valores que reflejan la misión de una aerolínea moderna: transportar pasajeros de forma rápida, segura y confiable.

2. Nombre de la Aplicación

Identificación del Proyecto

Nombre de la Aplicación: Pegaso Airlines: Plataforma Integral de Gestión Aeronáutica

Dominio: Transporte Aéreo Comercial

Motor de Base de Datos: MySQL 8.0+ (InnoDB)

Framework: Python 3.11 + Flask

3. Objetivo General

Diseñar y desarrollar una base de datos relacional transaccional (MySQL), que centralice, controle y valide las operaciones de la aerolínea ecuatoriana Pegaso. El sistema resuelve problemas críticos de la industria aeronáutica:

Concurrencia: Múltiples agentes emitiendo boletos simultáneamente sobre el mismo vuelo sin generar sobreventa no controlada, mediante bloqueos a nivel de fila (`SELECT ... FOR UPDATE`).

Integridad Referencial: 22 tablas normalizadas con restricciones `FOREIGN KEY`, `CHECK`, `UNIQUE` y `ENUM` que imposibilitan la inserción de datos inconsistentes.

Auditoría y Regulación: Triggers que registran automáticamente cada operación y prohíben la eliminación física de registros de pasajeros, cumpliendo con regulaciones de seguridad aeroportuaria.

4. Funcionalidades del Sistema

El sistema cubre el ciclo operativo completo de una aerolínea a través de 22 funcionalidades agrupadas en cuatro módulos.

4.1 Módulo de Reservaciones (7 funcionalidades)

#	Funcionalidad	Descripción
F01	Búsqueda de vuelos	Por origen/destino, fecha y número de pasajeros. Incluye búsqueda rápida por Ruta Maestra.
F02	Creación de PNR	Código alfanumérico único de 6 caracteres generado aleatoriamente.
F03	Emisión de e-tickets	Boletos electrónicos de 13 dígitos vinculados a la reservación.
F04	Selección de tarifa	Familias Light, Standard, Full y Flex con beneficios diferenciados.
F05	Autocompletado	Consulta en tiempo real (API REST) la tabla personas por número de cédula.
F06	Precios dinámicos	Cálculo basado en distancia de la ruta $\times$ factor de clase $\times$ factor de tarifa.
F07	Cancelación	Soft-delete: cambia estado a CANCELADA, libera asientos.

Cuadro 1: Funcionalidades del Módulo de Reservaciones

4.2 Módulo de Check-in y Equipaje (5 funcionalidades)

#	Funcionalidad	Descripción
F08	Búsqueda check-in	Por código PNR o apellido del pasajero.
F09	Proceso check-in	Validación de ventana de 48 horas y estado del boleto.
F10	Mapa de asientos	Visualización interactiva del avión. Solo muestra asientos de la clase pagada (Economy no puede seleccionar Business).
F11	Registro equipaje	Tag IATA de 10 dígitos, máximo 32 kg por pieza (regulación ocupacional). Valida contra franquicia de la tarifa.
F12	Pase de abordar	Boarding pass con código de barras, puerta, secuencia de embarque y clase.

Cuadro 2: Funcionalidades del Módulo de Check-in

4.3 Módulo de Vuelos y Tripulación (5 funcionalidades)

#	Funcionalidad	Descripción
F13	Programar vuelo	Supervisor selecciona la Ruta Maestra (no digita código ni hora) y solo elige la fecha y el avión.
F14	Estados de vuelo	Transiciones: Programado → Embarque → En Vuelo → Aterrizado → Arribado.
F15	Tripulación	Asignación de Comandante, Primer Oficial y Sobrecargos.
F16	Manifiesto	Lista completa de pasajeros por vuelo con asiento, clase y equipaje.
F17	Ocupación	Porcentaje de ocupación e ingresos por vuelo en tiempo real.

Cuadro 3: Funcionalidades del Módulo de Vuelos

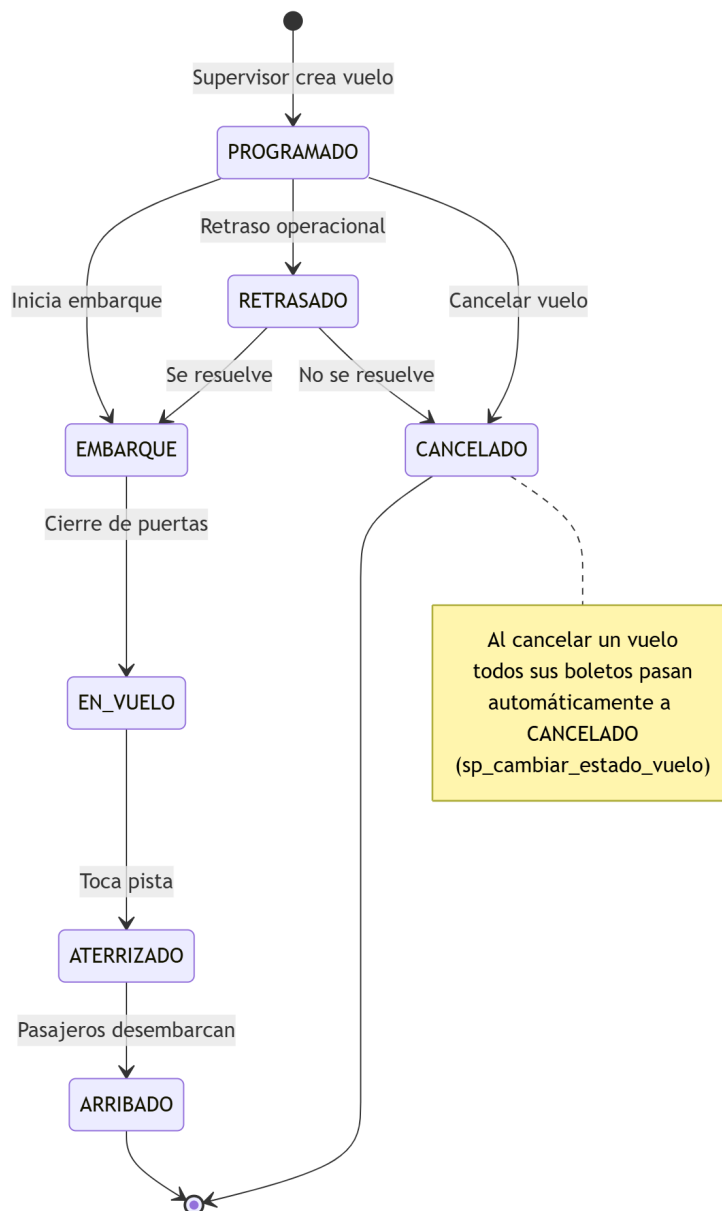


Figura 1: Máquina de estados y ciclo de vida de un Vuelo

4.4 Módulo de Administración (5 funcionalidades)

#	Funcionalidad	Descripción
F18	Gestión de flota	Alta de aeronaves con matrícula ecuatoriana (HC-), estado operativo.
F19	Gestión de personal	Registro de pilotos (con licencia), sobrecargos y agentes.
F20	Dashboard por rol	Panel diferenciado: Admin ve todo, Viajero solo sus reservas.
F21	Control de acceso	4 roles: Administrador, Supervisor, Agente, Viajero.
F22	Auditoría	Registro automático en tabla <code>auditoria</code> con datos JSON.

Cuadro 4: Funcionalidades del Módulo de Administración

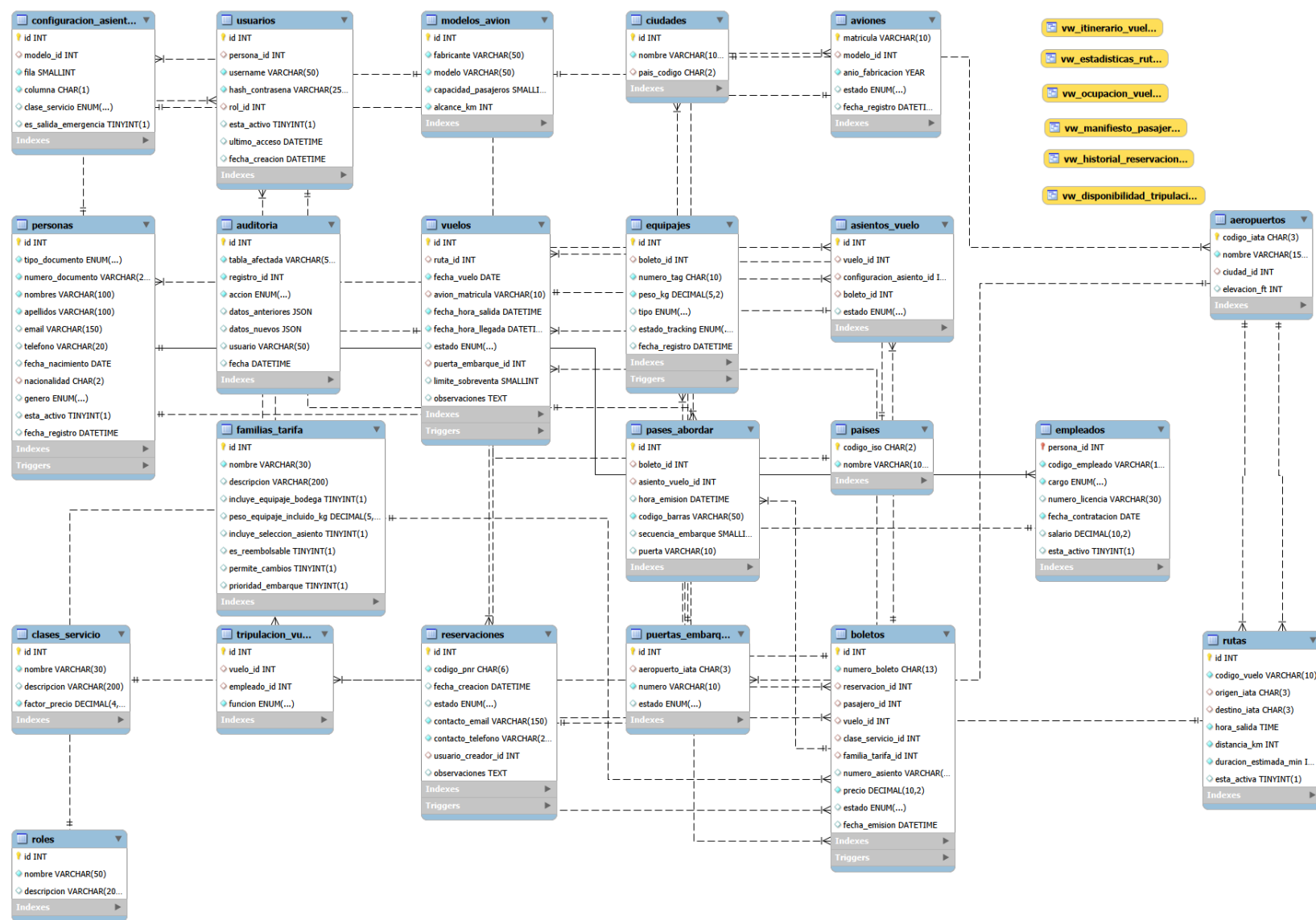
5. Modelo Físico de Base de Datos

La base de datos `pegaso_airlines` consta de **22 tablas** rigurosamente normalizadas hasta la Tercera Forma Normal (3FN), utilizando el motor **InnoDB** para garantizar transacciones ACID.

5.1 Diagrama Entidad-Relación

Nota sobre el diagrama

Muestra las 22 tablas con sus atributos, claves primarias, claves foráneas y cardinalidades.

Figura 2: Diagrama Entidad-Relación de la base de datos **pegaso_airlines** (Modelo Lógico completo)

5.2 Organización de las 22 Tablas

Grupo	Tabla	Propósito	PK
Base	países	Catálogo ISO 3166-1 alpha-2	codigo_iso
	ciudades	Ciudades con FK a país	id
	aeropuertos	Código IATA de 3 letras, elevación	codigo_iata
	puertas_embarque	Gates por aeropuerto con estado	id
Flota	modelos_avion	Airbus, Boeing, Embraer con alcance	id
	aviones	Matrícula ecuatoriana HC-XXX	matricula
	config_asientos	Layout: fila, columna, clase, emergencia	id
Personas	personas	Supertipo. Nunca se elimina (trigger)	id
	empleados	Subtipo: cargo, licencia, salario	persona_id
	roles	Admin, Supervisor, Agente, Viajero	id
	usuarios	Username + hash + rol + último acceso	id
Vuelos	rutas	Entidad maestra: código, hora, distancia	id
	vuelos	Instancia: ruta + fecha + avión + estado	id
	tripulacion_vuelo	Asignación crew con función	id
Comercial	clases_servicio	Economy, Premium, Business, First	id
	familias_tarifa	Light, Standard, Full, Flex	id
	reservaciones	PNR de 6 caracteres, estado	id
	boletos	E-ticket de 13 dígitos, precio	id
	asientos_vuelo	Mapa dinámico: disponible/ocupado	id
	equipajes	Tag IATA 10 dígitos, tracking	id
	pases_abordar	Código barras, secuencia, puerta	id
	auditoria	Log JSON de INSERT/UPDATE/DELETE	id

Cuadro 5: Inventario de las 22 tablas de `pegaso_airlines`

5.3 Restricciones y Constraints Destacados

- **PRIMARY KEYS (PK):** Definidas explícitamente en las 22 tablas del modelo para garantizar la identidad única de cada registro.
- **FOREIGN KEYS (FK) - 28 relaciones:** Destaca el uso estandarizado de **ON UPDATE CASCADE** en todas las llaves para la propagación de cambios, así como estrictos controles de eliminación:
 - **ON DELETE RESTRICT** en la mayoría de dependencias críticas (personas, vuelos, aeropuertos, aviones, etc.) para mantener el historial intacto y soportar el modelo de soft-delete.
 - **ON DELETE CASCADE** en dependencias directas que deben destruirse si desaparece el padre (`tripulacion_vuelo`, `boletos`, `asientos_vuelo`, `pases_abordar`).
 - **ON DELETE SET NULL** excepcionalmente en `asientos_vuelo(boleto_id)` para liberar el asiento si se cancela un boleto específico.

■ UNIQUE Compuestos (8 restricciones):

- `boletos(vuelo_id, numero_asiento)`: Impide que dos pasajeros tengan el mismo asiento.
- `puertas_embarque(aeropuerto_iata, numero)`: Evita duplicar el número de puerta en un mismo aeropuerto.
- `modelos_avion(fabricante, modelo)` y `configuracion_asientos(modelo_id, fila, columna)`.
- `rutas(origen_iata, destino_iata, hora_salida)`: Previene rutas idénticas a la misma hora exacta.
- `personas(tipo_documento, numero_documento, nacionalidad)`: Garantiza identidad única por documento.
- `tripulacion_vuelo(vuelo_id, empleado_id)`: Evita la doble asignación del mismo tripulante.
- `asientos_vuelo(vuelo_id, configuracion_asiento_id)`: Bloquea la duplicidad de la configuración espacial dentro de un mismo vuelo.

■ UNIQUE Simples (13 restricciones): Implementadas para garantizar identificadores de negocio irrepetibles en campos críticos como `codigo_vuelo`, `nombre` (para roles, clases de servicio y tarifas), `codigo_empleado`, `numero_licencia`, `username`, `codigo_pnr`, `numero_boleto`, `numero_tag` de equipaje, y `codigo_barras`.**■ CHECK Constraints (7 restricciones):**

- En vuelos: `fecha_hora_llegada > fecha_hora_salida`.
- En boletos: `precio >= 0`.
- En equipajes: `peso_kg > 0`.
- En modelos_avion: `capacidad_pasajeros > 0`.
- En rutas: `distancia_km > 0` y `duracion_estimada_min > 0`.
- En empleados: `salario >= 0`.

■ Valores por Defecto (DEFAULT): Implementados en la estructura para optimizar la inserción de datos y definir estados iniciales lógicos, cumpliendo con el diseño físico:

- **Fechas automáticas:** Uso de `DEFAULT CURRENT_TIMESTAMP` en tablas de auditoría, reservaciones y emisión de pases de abordar.
- **Estados operativos iniciales:** `estado ENUM(...) DEFAULT 'PROGRAMADO'` (vuelos), `DEFAULT 'DISPONIBLE'` (asientos y puertas), y `DEFAULT 'PENDIENTE'` (reservaciones).
- **Banderas lógicas:** `esta_activo BOOLEAN DEFAULT TRUE` aplicado en personas, empleados y usuarios para el correcto manejo de la política de soft-delete.

6. Implementación Funcional (Frontend)

El frontend fue desarrollado con **Python (Flask) + HTML5/CSS3/JavaScript puro**, siguiendo una arquitectura MVC con herencia de templates Jinja2. El diseño visual utiliza una paleta corporativa azul marino (#0F2B46) y dorado (#C9A84C), inspirada en aerolíneas latinoamericanas.

Stack Tecnológico

Backend:	Python 3.11 + Flask + Jinja2
Frontend:	HTML5, CSS3 (1,449 líneas), JavaScript ES6 (164 líneas)
Conexión BD:	mysql-connector-python con pool de 5 conexiones
Autenticación:	Sessions + werkzeug.security (SHA-256 / bcrypt)
Templates:	21 archivos HTML en 5 directorios

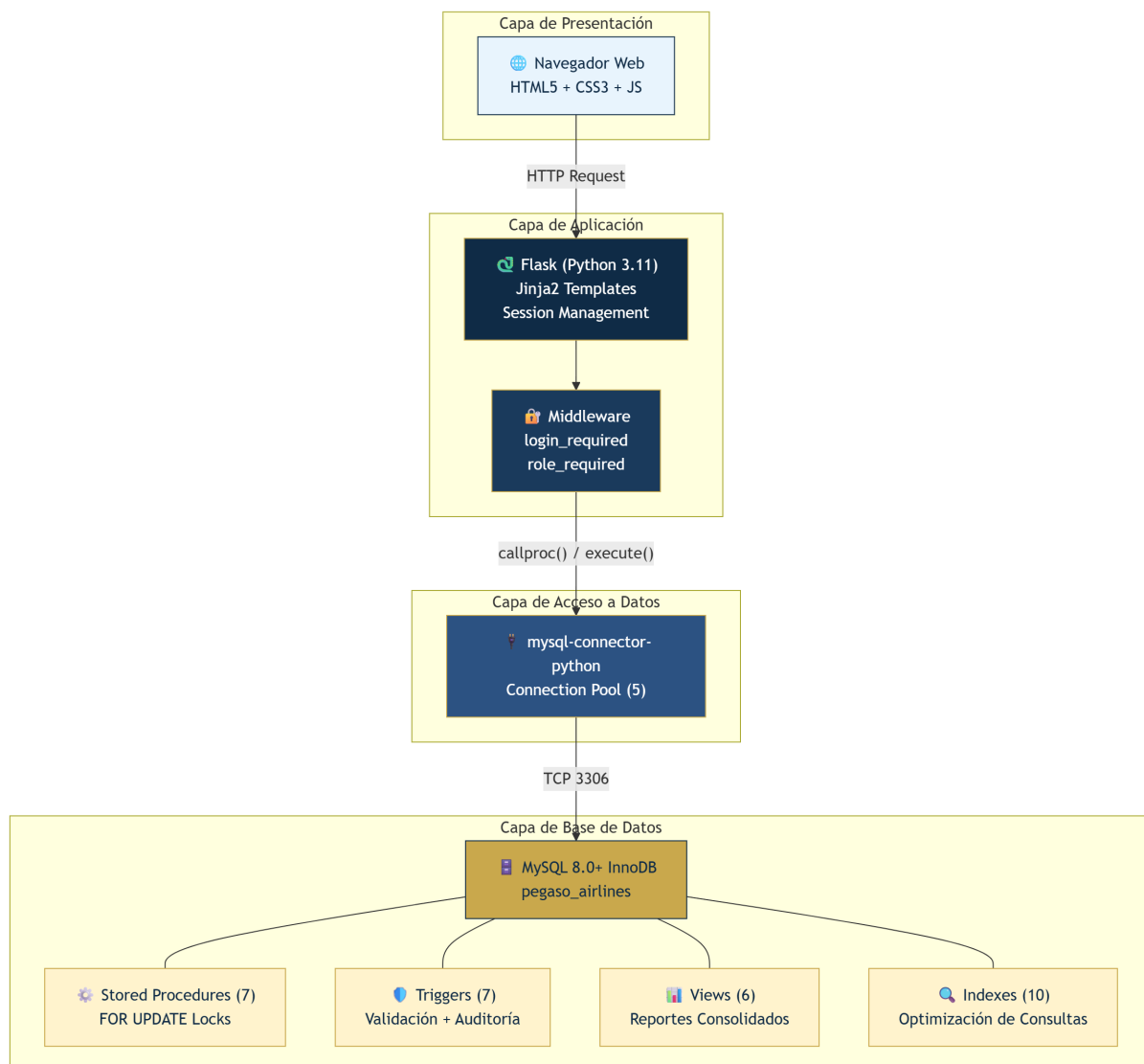


Figura 3: Arquitectura del Sistema y Patrón MVC

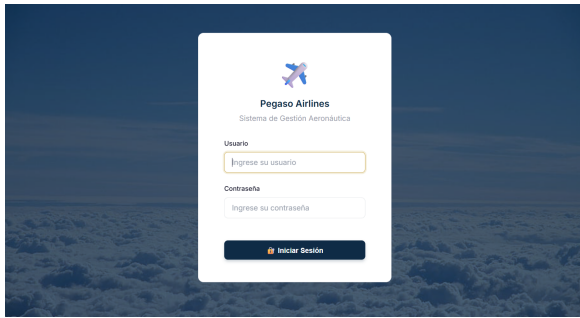


Figura 4: Pantalla de Acceso

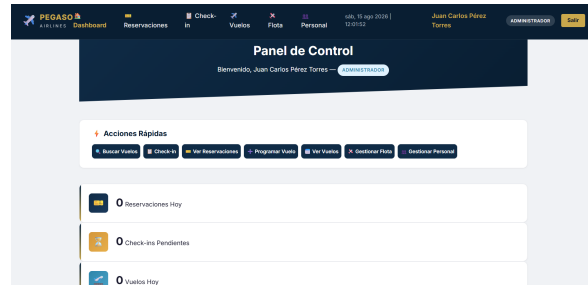


Figura 5: Dashboard principal diferenciado por Rol

6.1 CRUD 1: Gestión de Reservaciones

CRUD	Método	Ruta	Acción en la BD
Create	POST	/reservaciones/nueva	sp_crear_reservacion + sp_emitir_boleto (INSERT transaccional).
Read	GET	/reservaciones/{id}	SELECT con JOIN de 6 tablas para mostrar el detalle del PNR.
Update	POST	/reservaciones/nueva	Actualización de estado en BD: UPDATE reservaciones SET estado = 'CONFIRMADA' tras emisión de boleto.
Delete	POST	/reservaciones/cancelar/{id}	sp_cancelar_reservacion (Soft-Delete que ejecuta un UPDATE a CANCELADA).

Cuadro 6: Operaciones CRUD del módulo de Reservaciones

6.1.1 Cálculo Dinámico de Precios

El precio de cada boleto se calcula en el backend mediante la siguiente fórmula:

$$\text{Precio} = \underbrace{(30 + d \times 0,15)}_{\text{Precio base por distancia}} \times \underbrace{f_c}_{\text{Factor clase}} \times \underbrace{f_t}_{\text{Factor tarifa}}$$

Donde d es la distancia en kilómetros de la ruta, f_c es el factor de la clase de servicio (Economy = 1.00, Premium = 1.50, Business = 2.50, First = 4.00) y f_t varía según la familia tarifaria (Light = 1.0, Standard = 1.2, Full = 1.4, Flex = 1.6).

Ejemplo real del sistema

UIO → GYE (280 km): \$72.00 base × 1.00 (Economy) × 1.2 (Standard) = **\$86.40**
 UIO → MIA (2,880 km): \$462.00 base × 2.50 (Business) × 1.4 (Full) = **\$1,617.00**

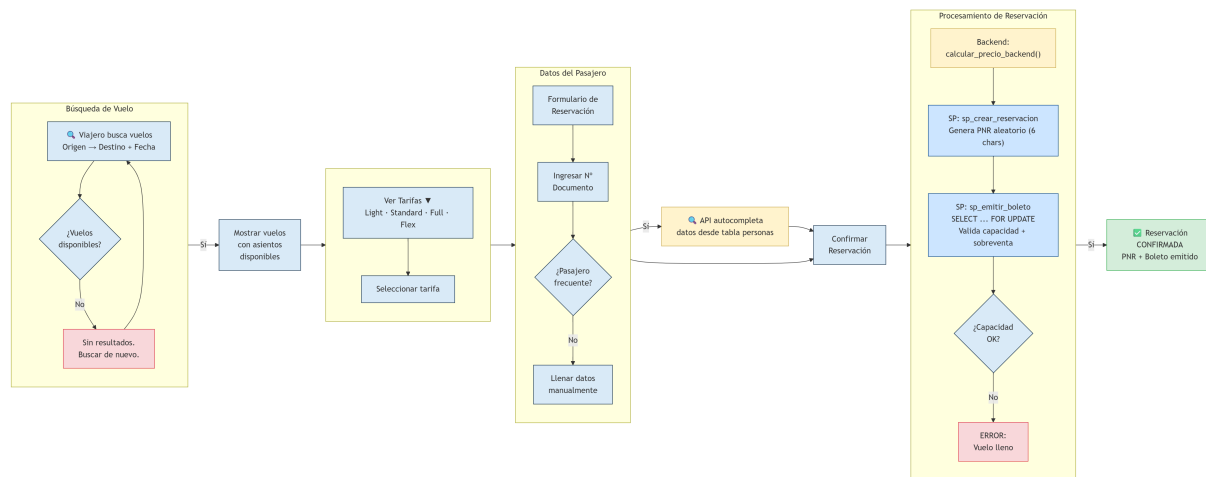


Figura 6: Flujo de proceso del Módulo de Reservas

6.2 CRUD 2: Check-in y Equipaje

CRUD	Método	Ruta	Acción en la BD
Create	POST	/checkin/proceso/{id}	sp_checkin_pasajero (check-in + pase)
Read	GET	/checkin/pase/{id}	SELECT con JOIN de 9 tablas
Update	POST	/checkin/equipaje/{id}	sp_registrar_equipaje
Delete	N/A	—	No permitido (regulación de seguridad)

Cuadro 7: Operaciones CRUD del módulo de Check-in

Validaciones de Seguridad Aeronáutica

- El mapa de asientos **filtra por clase**: un pasajero Economy no puede seleccionar asientos de Business (validado en frontend y en el SP).
- El peso máximo por pieza de equipaje de bodega es **32 kg** (regulación de salud ocupacional para estibadores, IATA).
- El equipaje se valida contra la **franquicia de la tarifa** (Light = 0 kg, Standard = 23 kg, Full = 46 kg, Flex = 64 kg).
- La secuencia de embarque se calcula **automáticamente** incrementando por vuelo.

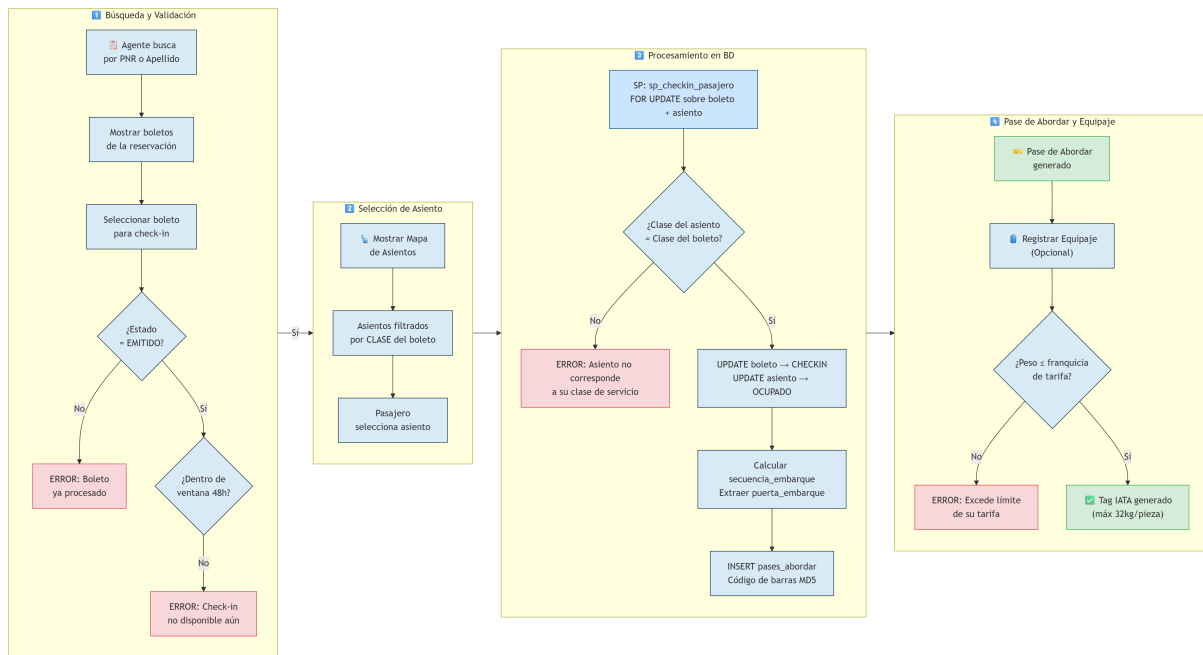


Figura 7: Flujo del proceso de Check-in y Equipaje

Carlos Luis López Mora

Doc: A1234567

Boleto: 0011234567890

PG101

UIO → GYE

27/07/2026 10:00

Mapa de Asientos

Disponible
 Ocupado
 Seleccionado
 Business

A	B	C	D	E	F
1A	1B	1C	1	1D	1E 1F
2A	2B	2C	2	2D	2E 2F
3A	3B	3C	3	3D	3E 3F
4A	4B	4C	4	4D	4E 4F
5A	5B	5C	5	5D	5E 5F
6A	6B	6C	6	6D	6E 6F
7A	7B	7C	7	7D	7E 7F
8A	8B	8C	8	8D	8E 8F
9A	9B	9C	9	9D	9E 9F

Figura 8: Selección de asientos (Validación estricta por Clase de Servicio)

Figura 9: Registro validando peso máximo de equipaje (32kg)

Figura 10: Pase de abordar con datos de puerta y secuencia

6.3 CRUD 3: Gestión de Vuelos y Tripulación

CRUD	Método	Ruta	Acción en la BD
Create	POST	/vuelos/nuevo	INSERT + trigger genera 180 asientos
Read	GET	/vuelos/{id}	Manifiesto con tripulación y pasajeros
Update	POST	/vuelos/estado/{id}	sp_cambiar_estado_vuelo
Delete	POST	/vuelos/tripulacion/eliminar	DELETE FROM tripulacion_vuelo

Cuadro 8: Operaciones CRUD del módulo de Vuelos

Distinción Ruta vs. Vuelo

El Supervisor **nunca digita** el código de vuelo ni la hora de salida. Al seleccionar la Ruta Maestra (ej. PG101 | Quito → Guayaquil | 10:00), el sistema autocompleta esos campos. El Supervisor solo elige la **fecha** y el **avión**, porque el Vuelo es simplemente una instancia de la Ruta en un día específico.

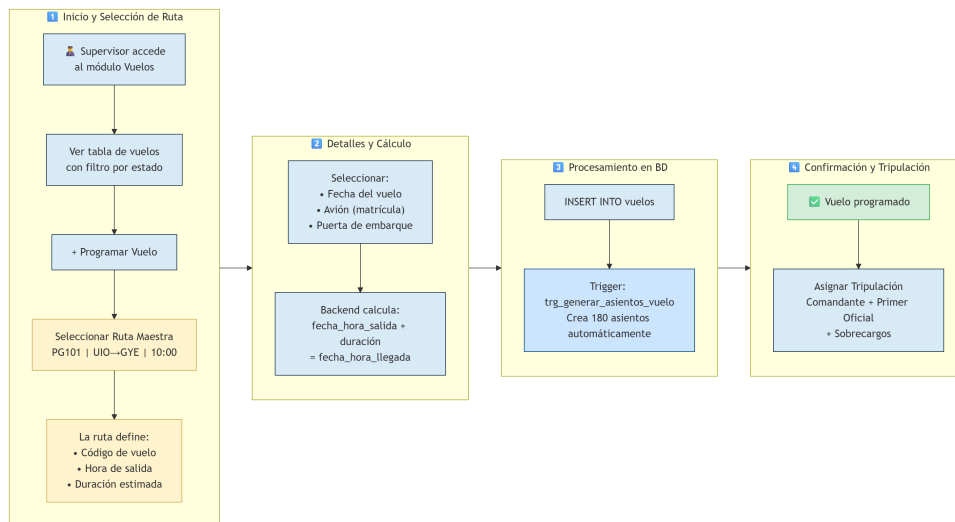


Figura 11: Flujo de instanciación de un Vuelo a partir de una Ruta Maestra

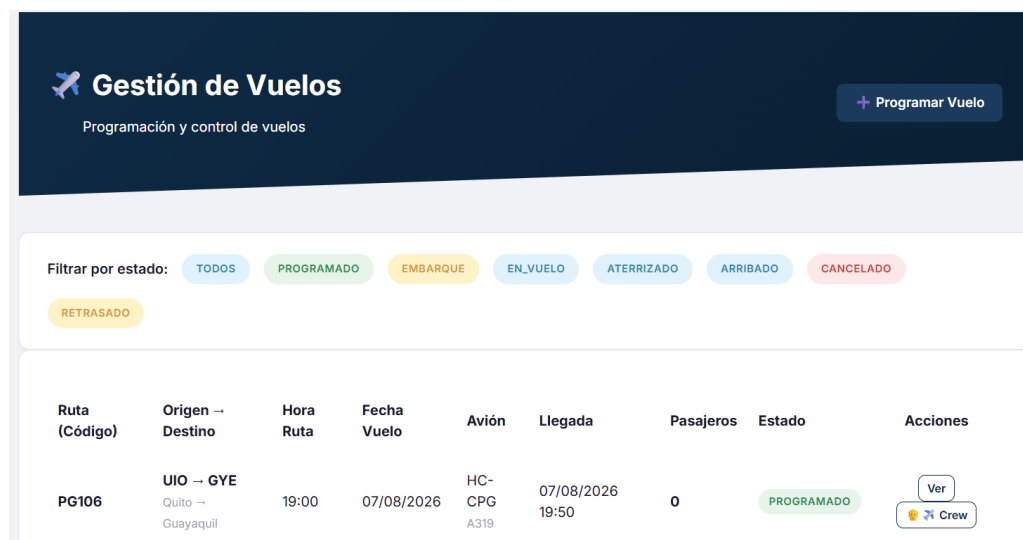


Figura 12: Interfaz del Supervisor para programar Vuelos

6.4 Módulo de Administración

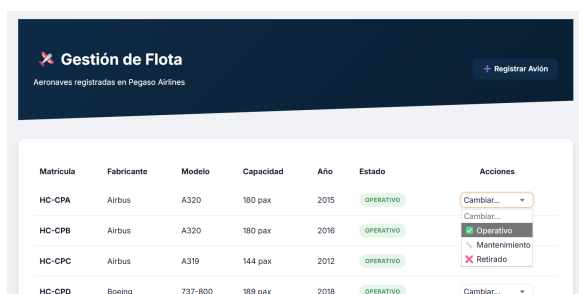


Figura 13: Gestión de Flota (Aviones y Estados)

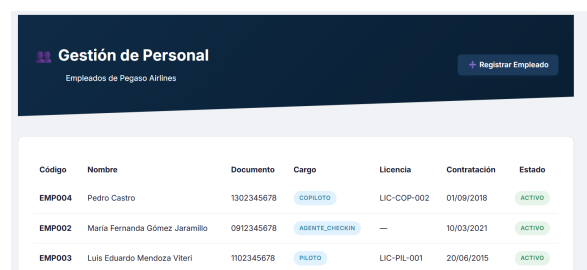


Figura 14: Gestión de Personal y Licencias

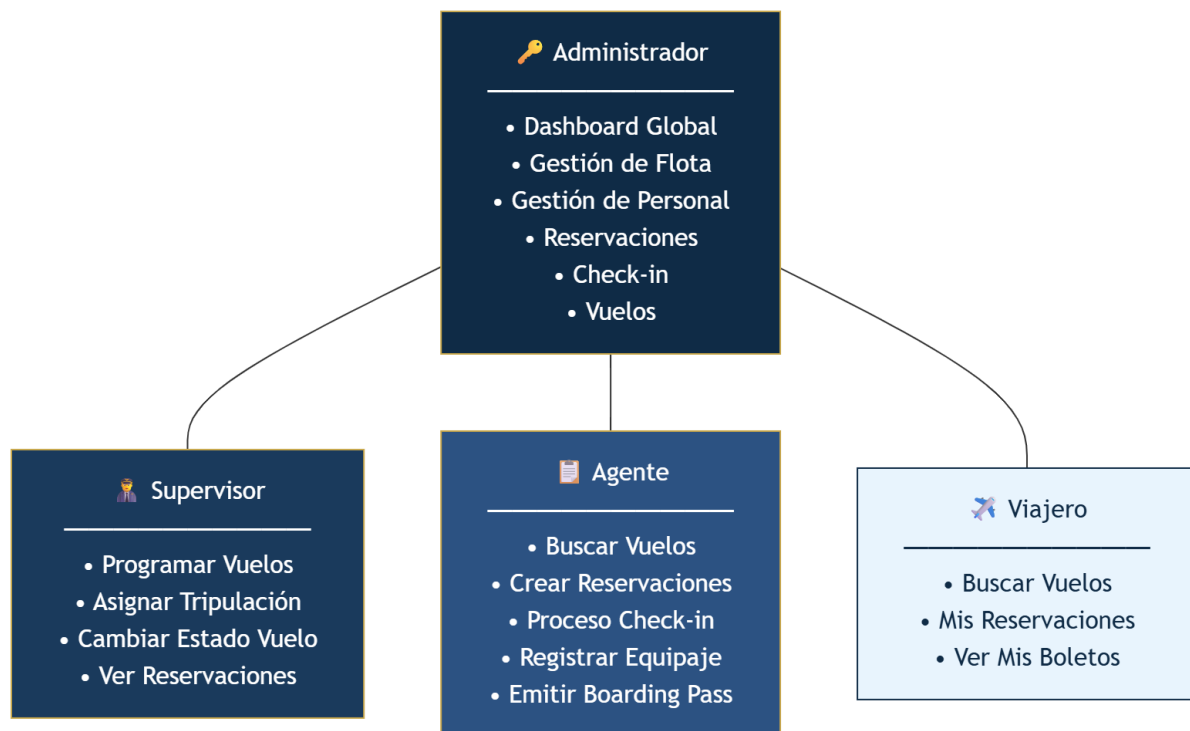


Figura 15: Jerarquía de roles y control de acceso del sistema

7. Video de Demostración Funcional

El video muestra las 3 funcionalidades anteriores en acción desde el frontend, seguidas inmediatamente de una consulta **SELECT** en MySQL Workbench para evidenciar que las transacciones y los bloqueos funcionaron correctamente en la base de datos.

Enlace del video: <https://youtu.be/PZ2f5dkD9Po>

8. Procedimientos Almacenados

Se implementaron **7 procedimientos almacenados** para soportar las operaciones críticas del sistema. Cumpliendo estrictamente con las exigencias del proyecto, todos estos procedimientos integran: **parámetros** de entrada (IN) y salida (OUT), **control transaccional** completo (START TRANSACTION / COMMIT / ROLLBACK), **control de errores** (DECLARE EXIT HANDLER FOR SQLEXCEPTION) y **control de bloqueos** por fila (SELECT ... FOR UPDATE) para garantizar la consistencia ante peticiones concurrentes.

Procedimiento	Propósito	FOR UPDATE	Params OUT
sp_crear_reservacion	Genera PNR único de 6 chars, valida email	No	PNR, resultado
sp_emitir_boleto	Emite e-ticket de 13 dígitos, valida capacidad + sobreventa	Sí	boleto, resultado
sp_checkin_pasajero	Check-in con asiento, valida clase, genera boarding pass	Sí	código barras, resultado
sp_registrar equipaje	Tag IATA, valida franquicia de tarifa y peso $\leq$ 32kg	No	tag, resultado
sp_cancelar_reservacion	Soft-delete: cancela reserva + boletos + libera asientos	Sí	resultado
sp_cambiar_estado_vuelo	Transición de estados, cancela boletos si vuelo cancelado	No	resultado
sp_asignar_tripulacion	Asigna crew validando duplicados	No	resultado

Cuadro 9: Resumen de los 7 procedimientos almacenados

8.0.1 Ejemplo: sp_emitir_boleto (Control de Sobreventa)

```

1  -- Bloqueo exclusivo sobre el registro del vuelo
2  SELECT estado, limite_sobreventa INTO v_estado_vuelo,
   v_limite_sobreventa
3  FROM vuelos WHERE id = p_vuelo_id FOR UPDATE;
4
5  -- Conteo de boletos activos
6  SELECT COUNT(*) INTO v_vendidos
7  FROM boletos WHERE vuelo_id = p_vuelo_id AND estado NOT IN ('CANCELADO')
   ;
8
9  -- Validacion de capacidad + politica de sobreventa
10 IF (v_vendidos >= (v_capacidad + v_limite_sobreventa)) THEN
11     SET p_resultado = 'ERROR: Capacidad maxima excedida.';
12     ROLLBACK;
13 END IF;

```

8.0.2 Ejemplo: sp_checkin_pasajero (Validación de Clase)

```

1  -- Obtener clase del boleto
2  SELECT REPLACE(UPPER(cs.nombre), ' ', '_')
3  INTO v_clase_boleto
4  FROM boletos b
5  JOIN clases_servicio cs ON b.clase_servicio_id = cs.id
6  WHERE b.id = p_boleto_id FOR UPDATE;
7
8  -- Obtener clase del asiento seleccionado
9  SELECT c.clase_servicio INTO v_clase_asiento
10 FROM asientos_vuelo av
11 JOIN configuracion_asientos c ON av.configuracion_asiento_id = c.id
12 WHERE av.id = p_asiento_vuelo_id FOR UPDATE;
13
14 -- Validacion cruzada: la clase debe coincidir
15 IF v_clase_boleto != v_clase_asiento THEN
16     SET p_resultado = 'ERROR: El asiento no corresponde a su clase.';
17     ROLLBACK;
18 END IF;

```

9. Disparadores (Triggers)

Se implementaron **7 triggers** clasificados en tres categorías funcionales:

#	Trigger	Evento	Timing	Justificación
1	trg_audit_reserv_ins	INSERT reservaciones	AFTER	Trazabilidad comercial: registra PNR y estado
2	trg_audit_reserv_upd	UPDATE reservaciones	AFTER	Cumplimiento regulatorio: estado anterior vs. nuevo
3	trg_validar_capacidad	INSERT boletos	BEFORE	Integridad: bloquea venta si se excede capacidad
4	trg_audit_boleto_upd	UPDATE boletos	AFTER	Ciclo de vida: EMITIDO → CHECKIN → ABORDADO
5	trg_validar_peso	INSERT equipajes	BEFORE	Regulación IATA: ≤ 32 kg bodega, ≤ 10 kg cabina
6	trg_proteger_persona	DELETE personas	BEFORE	Seguridad aeroportuaria: impide eliminación física
7	trg_generar_asientos	INSERT vuelos	AFTER	Operacional: genera mapa de 180 asientos automáticamente

Cuadro 10: Inventario de los 7 triggers

9.0.1 Ejemplo: Protección contra eliminación física

```

1 CREATE TRIGGER trg_proteger_eliminacion_persona
2 BEFORE DELETE ON personas
3 FOR EACH ROW
4 BEGIN
5     SIGNAL SQLSTATE '45000'
6     SET MESSAGE_TEXT = 'No se permite la eliminacion fisica
7     de personas. Use desactivacion (esta_activo = FALSE)';
8 END;
```

Esta restricción es crítica en la industria aeronáutica: las autoridades de aviación civil pueden requerir el historial completo de un pasajero años después de su viaje. El sistema utiliza **eliminación lógica** (campo `esta_activo = FALSE`) en lugar de `DELETE`.

10. Consultas SQL y Vistas

10.1 Vistas Implementadas (6)

#	Vista	Propósito	JOINS
1	<code>vw_itinerario_vuelos</code>	Vuelos con ruta, avión, puerta y asientos disponibles calculados en tiempo real	7
2	<code>vw_manifiesto_pasajeros</code>	Lista de pasajeros por vuelo con boleto, asiento y cantidad de equipajes	4
3	<code>vw_ocupacion_vuelos</code>	Porcentaje de ocupación, ingresos totales y pasajeros chequeados	4
4	<code>vw_historial_reservaciones</code>	Historial de PNRs con pasajeros, boletos y vuelos asociados	5
5	<code>vw_disponibilidad_tripulacion</code>	Empleados activos disponibles para asignación	2
6	<code>vw_estadisticas_rutas</code>	Frecuencia, ocupación promedio e ingresos por ruta	3

Cuadro 11: Vistas del sistema con complejidad de JOINS

10.1.1 Ejemplo: Vista de Ocupación de Vuelos

```

1 CREATE OR REPLACE VIEW vw_ocupacion_vuelos AS
2 SELECT
3     v.id AS vuelo_id,
4     r.codigo_vuelo AS numero_vuelo,
5     m.capacidad_pasajeros AS capacidad_total,
6     COUNT(b.id) AS boletos_vendidos,
7     SUM(CASE WHEN b.estado IN ('CHECKIN', 'ABORDADO', 'COMPLETADO')
8         THEN 1 ELSE 0 END) AS pasajeros_chequeados,
9     ROUND((COUNT(b.id) / m.capacidad_pasajeros) * 100, 2)
10    AS porcentaje_ocupacion,
```

```

11      SUM(b.precio) AS ingresos_totales
12 FROM vuelos v
13 JOIN rutas r ON v.ruta_id = r.id
14 JOIN aviones a ON v.avion_matricula = a.matricula
15 JOIN modelos_avion m ON a.modelo_id = m.id
16 LEFT JOIN boletos b ON v.id = b.vuelo_id
17     AND b.estado NOT IN ('CANCELADO', 'NO_SHOW')
18 GROUP BY v.id, r.codigo_vuelo, m.capacidad_pasajeros;

```

10.2 Índices Optimizados (10) y Consultas de Aplicación

Para garantizar la máxima eficiencia y cumplir estrictamente con los requerimientos operativos de la rúbrica, a continuación se justifican los 10 índices creados, junto con la consulta SQL exacta donde el motor InnoDB los utiliza:

Índice y Campos	Justificación del orden	Consulta SQL Exacta de uso en el sistema
idx_vuelos_fecha_estado (fecha, estado)	Filtro Rango + Exacto: La fecha filtra el volumen masivo y estado refina.	SELECT * FROM vuelos WHERE DATE(fecha_hora_salida) = %s AND estado = 'PROGRAMADO'
idx_boletos_reservacion (reservacion_id)	FK más consultada: Vital para traer el detalle de boletos de un PNR.	SELECT * FROM boletos WHERE reservacion_id = %s
idx_boletos_vuelo_estado (vuelo_id, estado)	Agrupación y Exclusión: Filtra por vuelo y excluye cancelados (sobreventa).	SELECT COUNT(*) FROM boletos WHERE vuelo_id = %s AND estado NOT IN ('CANCELADO')
idx_boletos_pasajero (pasajero_id)	Búsqueda Directa FK: Recupera el historial de viajes de un viajero.	SELECT * FROM boletos b JOIN personas p ON b.pasajero_id = p.id WHERE p.numero_documento = %s
idx_personas_documento (tipo_doc, num_doc)	Discriminación: El tipo de documento evita cruces (Pasaporte vs Cédula).	SELECT id FROM personas WHERE tipo_documento = %s AND numero_documento = %s
idx_reservaciones_pnr (codigo_pnr)	Búsqueda Exacta: Punto de entrada principal para el proceso de Check-in.	SELECT id FROM reservaciones WHERE codigo_pnr = %s
idx Equipajes boleto (boleto_id)	Búsqueda FK: Trae las maletas registradas para imprimir el pase.	SELECT * FROM equipajes WHERE boleto_id = %s ORDER BY fecha_registro
idx_tripulacion_vuelo (vuelo_id, empleado_id)	Búsqueda Compuesta: Verifica si el crew ya está asignado al vuelo.	SELECT COUNT(*) FROM tripulacion_vuelo WHERE vuelo_id = %s AND empleado_id = %s
idx_vuelos_ruta (ruta_id)	Agrupación FK: Usado en la vista de estadísticas de rentabilidad por ruta.	SELECT * FROM rutas r LEFT JOIN vuelos v ON r.id = v.ruta_id GROUP BY r.id, ruta
idx_auditoria_tabla_fecha (tabla, fecha)	Filtro Rango: Optimiza reportes de auditoría recientes por entidad.	SELECT * FROM auditoria WHERE tabla_afectada = 'reservaciones' ORDER BY fecha DESC

Cuadro 12: Índices optimizados, justificación técnica y mapeo de consultas SQL reales

11. Organización y Documentación del Proyecto

11.1 Estructura de Carpetas

```

1 pegaso/
2 |-- database/
3 |   |-- 01_pegaso_schema.sql           (22 tablas, PKs, FKs, CHECKs)
4 |   |-- 02_pegaso_indexes.sql         (10 indices con justificacion
5 |   )
6 |   |-- 03_pegaso_views.sql           (6 vistas)
7 |   |-- 04_pegaso_procedures.sql       (7 stored procedures)
8 |   |-- 05_pegaso_triggers.sql         (7 triggers)
9 |   |-- 06_pegaso_seed.sql             (data semilla Ecuador)
10 |   |-- 07_pegaso_pruebas.sql          (script de demo en vivo)
11 |-- templates/                        (21 archivos Jinja2)
12 |   |-- layout.html                   (plantilla base)
13 |   |-- dashboard.html                 (panel por rol)
14 |   |-- reservaciones/                 (5 templates)
15 |   |-- checkin/                       (5 templates)
16 |   |-- vuelos/                        (4 templates)
17 |   |-- admin/                         (4 templates)
18 |-- static/
19 |   |-- css/style.css                  (1,449 lineas)
20 |   |-- js/app.js                      (164 lineas)
21 |   |-- fondo-avion.jpg                (login background)
22 |-- app.py                            (996 lineas, 19 rutas)
23 |-- requirements.txt                   (4 dependencias)
24 |-- .env                              (configuracion MySQL)

```

11.2 Convenciones de Nomenclatura

Elemento	Convención	Ejemplo
Tablas	snake_case plural	reservaciones, pases_abordar
PKs	id o clave natural	codigo_iata, matricula
FKs	fk_tabla_ref	fk_vuelos_ruta
Índices	idx_tabla_campos	idx_vuelos_fecha_estado
SPs	sp_accion	sp_crear_reservacion
Triggers	trg_accion_tabla	trg_validar_peso equipaje
Vistas	vw_descripcion	vw_ocupacion_vuelos

Cuadro 13: Convenciones de nomenclatura SQL

11.3 Data Semilla

La base de datos se inicializa con datos coherentes del contexto ecuatoriano:

- **9 países** de la región (Ecuador, Colombia, Perú, EE.UU., México, Panamá, Brasil, Chile, Argentina)

- **14 ciudades** con **14 aeropuertos** reales (UIO, GYE, CUE, GPS, MEC, LOH, BOG, LIM, MIA, MEX, PTY, GRU, SCL, EZE)
- **4 modelos de avión:** Airbus A320 (180 pax), A319 (144), Boeing 737-800 (189), Embraer E190 (100)
- **8 aviones** con matrícula ecuatoriana (HC-CPA a HC-CPH)
- **180 asientos** por A320: filas 1-3 Business, 4-12 Premium Economy, 13-30 Economy
- **8 rutas** operadas (4 frecuencias diarias UIO-GYE + rutas internacionales)
- **50 vuelos** pre-programados para julio y agosto 2026

12. Conclusiones

El proyecto Pegaso Airlines demuestra la aplicación de los conceptos de Sistemas de Bases de Datos en un dominio de alta criticidad operativa. Los principales logros técnicos incluyen:

1. **Integridad transaccional:** Los 7 procedimientos almacenados garantizan que ninguna operación quede en estado inconsistente, incluso ante fallos o concurrencia.
2. **Seguridad multinivel:** Desde el frontend (roles, sesiones) hasta el backend (triggers, constraints), cada capa del sistema valida de manera independiente las reglas de negocio.
3. **Realismo aeronáutico:** El sistema refleja fielmente las operaciones de una aerolínea comercial: códigos PNR estándar, tags IATA de equipaje, límites regulatorios de peso (32 kg), control de sobreventa, clases diferenciadas y auditoría completa.
4. **Arquitectura escalable:** La separación entre Ruta (entidad maestra) y Vuelo (instancia) permite escalar a múltiples frecuencias diarias sin redundancia de datos.